

- Command 5: This command prints the value of number `pi` as stored in the `math` Erlang module. The result is a float number.
- Command 6: The `q()` function is the easiest way to exit the Erlang shell. The `q()` function quits everything Erlang is doing. If you are working locally, then there is no problem but if you are working on a remote system, you better quit by typing `Ctrl-G` and then `q`. The reason is that you may shut down the Erlang runtime on the remote machine when you quit using `q()`!

You are now ready to learn more interesting things about Erlang so keep reading, but it would also be a good idea to first experiment with the Erlang shell on your own.

Getting to know Erlang

Erlang was developed at Ericsson in order to serve the switching of telephone equipment. Although it looks like a special purpose programming language, it is not. Actually, if you are using a NoSQL data store such as *CouchDB* or *Riak*, you are already using Erlang without knowing it!

The first thing that you need to do is forget everything you know about classes, variables and assignment of variables. You will need to think more in terms of pattern matching, pathways and message passing. This may sound weird, but it's much more fun to work in this way because of the power it gives you!

Some of the main characteristics of Erlang are:

- It makes difficult things easy.
- It makes easy things difficult.
- Erlang was not designed for big data, but was designed for massive concurrency.
- It is a great language for fault tolerance and distributed applications.
- It concentrates more on data flow.
- You should design your programs based on recursion and message passing.
- Erlang can program Web applications.
- The syntax of Erlang does not look like a C-based language.
- *Variables* start with capital letters in Erlang. Words starting with lower case letters are *atoms*.
- *Atoms* are literals, constants with their own name for value.
- The `dbg` module and the `erlang:trace` functions are extremely helpful tools for debugging your code.
- Erlang allows you to build applications that tolerate failure and restore their functionality.
- Erlang comes with its own DBMS called *Mnesia*!
- You can easily create lightweight HTTP servers using the *MochiWeb* module.

Please do not forget that Erlang cannot solve every programming problem equally well — you may have to use another programming language for a specific type of software. Always use the best tool for the job.

The Erlang syntax may look strange at first but if you start writing Erlang programs, you will stop thinking about its syntax in less than a week! Your first exposure to Erlang may confuse or

frustrate you, while also leaving you impressed and excited!

Erlang uses two kinds of numbers, integers and floats. 1 is an integer, whereas 2.0 is a floating point number. Integer numbers are always precise but floating-point numbers may get truncated a bit. When you are using floats, you should always have a number to the left of the decimal point. In other words, you cannot write `.6` but you should write `0.6`.

When declaring a variable inside the Erlang shell, you cannot change its value, as the following example shows:

```
1> MyVar = 12.
12
2> MyVar = MyVar + 5.
** exception error: no match of right hand side value 17
3> MyVar = 5.
** exception error: no match of right hand side value 5
```

Erlang interprets `'MyVar = MyVar + 5'` as `"12 = 12 + 5"` which is obviously wrong. Erlang uses the *single assignment model*, which allows each variable to be assigned a value only once in a given session or context.

When you are using the Erlang shell (the `erl` command), every command is interpreted before being executed. When you want Erlang to compile a file, it converts the code into a BEAM file. A key part of the Erlang Runtime System (ERTS) is the BEAM processor. BEAM stands for Bodgan's Erlang Abstract Machine, which is a virtual machine that interprets optimised BEAM code.

The built-in line editor of `erl` is a subset of *Emacs*.

The Erlang shell is the perfect place to test most of your stuff before creating a module.

What is the Erlang OTP framework?

OTP stands for Open Telecom Platform and forms Erlang's open source libraries and tools. Despite its name, OTP is about much more than just telephones.

It is the most important and vital part of Erlang. If one third of Erlang's greatness comes from its concurrency and distribution, and another third comes from its error handling capabilities, then the OTP framework comprises the last third.

OTP was designed for building projects with big teams. It has been rewritten twice before, so it is now in its third version. It is about taking all the generic components, putting them into libraries, making sure they work well and reliably, and then reusing that code when possible. So, what is left to be done? The programmer has to only deal with things that will always change from application to application.

If you are going to write real world Erlang software, you will eventually have to learn and master the OTP framework.

A more advanced Erlang example

When creating programs, Erlang programmers put their code in *modules*, which help you organise, store and share code easily and efficiently and is the preferred way of writing